

Application Development Project Report

1. Student Details

Name: Priyavart

Roll Number: 23f2005399

Email: 23f2005399@ds.study.iitm.ac.in

About Me:

I am a BS (Data Science) student who enjoys building practical web applications. I like learning by implementing complete, end-to-end systems with clean data models and clear user flows. This project allowed me to explore full-stack development with Flask REST APIs, Vue.js reactive frontends, and background job orchestration using Celery and Redis.

2. Project Details

Project Title: Trail Sync – Trekking Management Application (MAD-II)

Problem Statement:

Adventure organisations require efficient systems to manage trekking activities involving trek organisers, staff, and participants. Currently, many trekking groups rely on spreadsheets, phone calls, or manual coordination, which makes it difficult to manage trek approvals, participant registrations, slot availability, and booking tracking. The goal was to build a web-based Trekking Management Application that allows Admin, Trek Staff (Guides), and Users (Trekking) to interact with the system based on their roles.

Approach:

The application is implemented with a decoupled architecture: a **Flask REST API** backend and a **Vue.js** single-page application frontend. Role-based access is enforced via **JWT tokens** stored in HTTP-only cookies. The backend exposes resource endpoints consumed by Axios calls in the frontend. Celery workers handle asynchronous and scheduled tasks (daily reminders, monthly admin reports, CSV export), while Redis serves both as a message broker and as a caching layer for frequently accessed trek listings. SQLite is used as the local relational database via SQLAlchemy ORM, with all tables and the default admin account created programmatically on startup.

3. AI/LLM Declaration

I used ChatGPT (approx. 10–12%) to assist with CSS styling, general aesthetics, and minor JavaScript helper functions. The usage was limited to front-end polishing and debugging layout issues, without relying on it for core application structure. All core implementation logic, debugging, integration, and verification were carried out independently.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask 3.1	Core backend REST API framework and application entry point.
SQLAlchemy / Flask-SQLAlchemy	ORM for data modelling and SQLite persistence.
SQLite	Lightweight local relational database (created programmatically).
PyJWT / Flask-Login	JWT-based token authentication and session management.
Vue.js 3 (CDN)	Reactive SPA frontend with component-based UI and Vue Router.
Bootstrap 5	Responsive HTML generation and UI styling.
Axios	HTTP client for asynchronous API communication from the frontend.
Celery 5.4	Distributed task queue for scheduled and user-triggered async jobs.
Redis	Message broker for Celery and in-memory cache for API responses.
Jinja2	Templating for the HTML entry-point and email notification bodies.
Werkzeug	Password hashing and WSGI utilities.
xhtml2pdf	Monthly activity report PDF generation for Admin.
Chart.js	Trek participation analytics and booking trend visualisations.
Vite	Frontend build tooling and development server.

5. Database Schema / ER Design

Tables:

- users** — Unified account table for all roles (*id, name, email, password_hash, role, phone, city, emergency, bio, blood_group, fitness_level, active, blacklisted, registered_at*).
- trek_routes** — Master catalogue of trekking destinations (*id, trek_code, name, location, difficulty, duration, distance, latitude, longitude, active*).
- treks** — Scheduled trek batch instances (*id, trek_route_id, batch_code, name, location, difficulty, start_date, end_date, slots, price, status, staff_id, image_url*).
- bookings** — Trekker reservations for a batch (*id, user_id, trek_id, booked_on, status, payment_status, payment_method, booking_price, amount_paid, refund_amount*).
- staff_profiles** — Guide-specific qualifications extending the users table (*id, user_id, designation, experience_years, certifications, skills, languages, completed_treks_count, status*).
- booking_checklist_items** — Packing checklist items per booking (*id, booking_id, item_name, category, is_completed*).

7. **support_tickets** — User/staff support requests (*id, user_id, subject, message, status, resolution_message, created_at*).
8. **chat_messages** — Trek group chat messages (*id, trek_id, sender_id, sender_name, sender_role, message_text, is_announcement, created_at*).
9. **trek_guide_items** — Custom gear recommendations added by guides per batch.
10. **trek_group_settings** — Chat room locked/unlocked state per trek.
11. **user_announcement_reads** — Tracks last-read announcement per user per trek (unread badges).

Relationships:

- One-to-One **users** **staff_profiles** (guide extension)
- One-to-Many **trek_routes** **treks** (route has multiple scheduled batches)
- One-to-Many **users** (staff) **treks** (guide leads multiple batches)
- One-to-Many **users** (trekker) **bookings**
- One-to-Many **treks** **bookings**
- One-to-Many **bookings** **booking_checklist_items**
- One-to-Many **users** **support_tickets**
- One-to-Many **treks** **chat_messages**

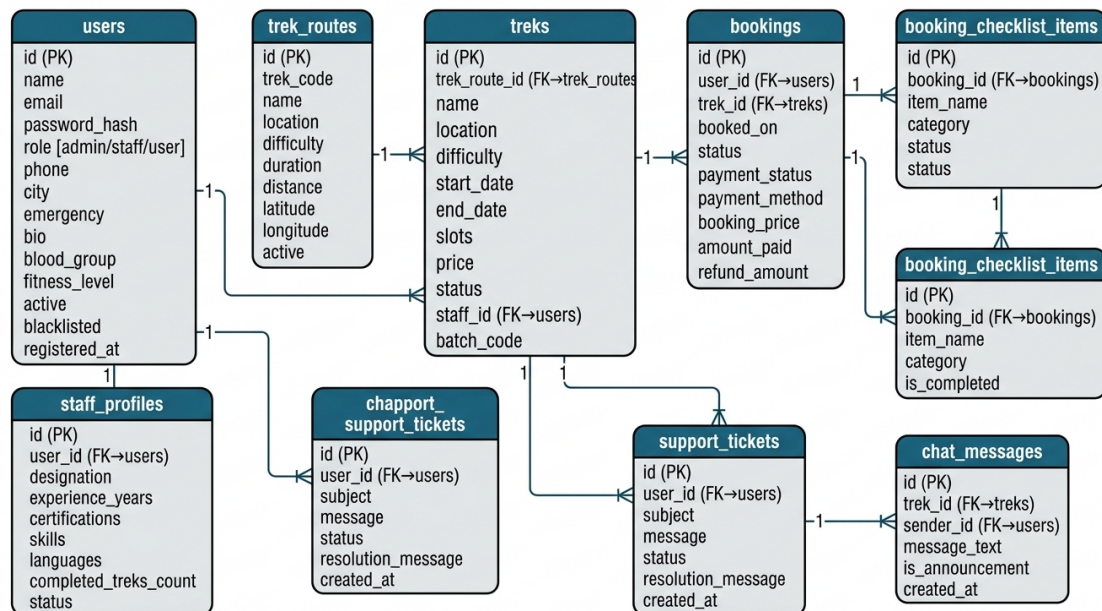


Figure 1: Entity-Relationship Diagram for Trail Sync (generated from database models)

6. API Resource Endpoints

The complete API specification is detailed in the OpenAPI 3.0.3 schema file included separately in the submission ZIP as `api.yaml`.

Endpoint	Method	Description
<code>/api/auth/register</code>	POST	Register a new trekker account.
<code>/api/auth/login</code>	POST	Authenticate user and set JWT cookie.
<code>/api/auth/check-availability</code>	GET	Check email/phone availability.
<code>/api/auth/logout</code>	POST	Clear the session token cookie.
<code>/api/public/treks</code>	GET	Fetch open scheduled treks (cached).
<code>/api/public/trek_routes</code>	GET	Fetch master trek route catalogue.
<code>/api/user/dashboard_data</code>	GET	Return trekker dashboard payload.
<code>/api/bookings/book</code>	POST	Book a trek batch; initialises packing checklist.
<code>/api/bookings/cancel/{booking_id}</code>	POST	Cancel a booking and initiate refund.
<code>/api/bookings/{booking_id}/checklist</code>	GET	Fetch packing checklist for booking.
<code>/api/bookings/checklist/toggle</code>	POST	Toggle completion state of a checklist item.
<code>/api/bookings/pay/{booking_id}</code>	POST	Simulate payment for a booking.
<code>/api/user/export</code>	POST	Trigger async CSV export of history via Celery.
<code>/api/user/profile</code>	POST	Update trekker profile.
<code>/api/user/tickets</code>	GET, POST	Fetch / submit support tickets.
<code>/api/staff/dashboard_data</code>	GET	Fetch assigned treks and participant lists.
<code>/api/staff/password</code>	POST	Update guide password.
<code>/api/staff/treks/slots/{trek_id}</code>	POST	Update slot capacity for a trek batch.
<code>/api/social/chat/{trek_id}</code>	GET	Fetch chat room messages for a trek group.

/api/social/chat/send	POST	Send a text message to a group chat room.
/api/social/chat/announcement	POST	Post guide announcement in chat.
/api/admin/<resource>	GET, POST, PUT, DELETE	Admin CRUD endpoints.

7. Architecture and Features

Architecture Overview:

- `backend/app.py` – Flask entry point; initialises DB, registers Blueprints.
- `backend/models/models.py` – SQLAlchemy ORM models.
- `backend/routes/` – Blueprints for auth, admin, staff, user, social chat, and public catalog.
- `backend/tasks.py` – Celery tasks (reminders, reports, CSV export).
- `backend/redis_utils.py` – Redis cache helper.
- `frontend/src/` – Vue.js SPA components, routing, and views.

Implemented Core Features:

- Unified user model supporting Admin, Trek Staff, and Trekker roles.
- Role-based access control and token authentication (JWT).
- Admin dashboard: routes/batches CRUD, staff assignment, user blacklisting, booking tracking.
- Guide dashboard: assigned treks list, slot capacity updates, participant tracking.
- Trekker dashboard: browse/filter treks, payment simulation, packing checklist, bookings list.
- Prevent overbooking and duplicate bookings; restrict booking to Open treks.

Additional / Extended Features:

- Trek group chat rooms with guide announcements and Celery-driven emails.
- Interactive packing checklist with custom gear recommended by guides.
- User-triggered booking history CSV export emailed asynchronously.
- Daily trek reminders via Celery Beat scheduled jobs.
- Monthly admin activity reports generated as PDF (xhtml2pdf) and emailed.
- Redis caching on public listings; Chart.js graphs for statistics.
- Responsive layout with Bootstrap 5.

8. Video Presentation

Link: <https://drive.google.com/file/d/1Se5SuAStxNiAvd6MD0bHMuu9lQLuzgYy/view?usp=sharing>